MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 11 — Les Ensembles (Sets)

Unicité, rapidité, logique mathématique

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

9 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 11 — Les Ensembles (Sets) **2**
0.1 Créer un ensemble 2
0.2 Propriétés fondamentales des sets 3
0.3 Ajouter et supprimer des éléments 3
0.4 Les opérations ensemblistes 4
0.5 Les relations entre ensembles 5
0.6 La set comprehension 5
0.7 Le frozenset — Ensemble immuable 6
0.8 Cas pratiques — Dédoublonnage et comparaisons 7
0.9 Ce qui m'a le plus surpris dans ce chapitre 8
0.10 Résumé du Chapitre 11 8

Chapitre 11 — Les Ensembles (Sets)

À retenir

Un ensemble (`set`) est une collection **non ordonnée** d'éléments **uniques**. Pas de doublons, pas d'index — mais une puissance mathématique redoutable pour les opérations ensemblistes : union, intersection, différence. Quand on veut travailler avec des **éléments distincts** ou comparer des collections, le set est l'outil parfait.

Créer un ensemble

Un ensemble est délimité par des **accolades** `{}` — comme un dictionnaire, mais sans les valeurs.

Créer des ensembles

```python
# Avec accolades
fruits = {"pomme", "banane", "cerise", "mangue"}
print(fruits)   # ordre non garanti !

# Avec set()
nombres = set([1, 2, 3, 4, 5])
print(nombres)   # {1, 2, 3, 4, 5}

# Depuis une chaine
lettres = set("python")
print(lettres)   # {'p', 'y', 't', 'h', 'o', 'n'}

# Les doublons sont AUTOMATIQUEMENT supprimes
avec_doublons = {1, 2, 2, 3, 3, 3, 4}
print(avec_doublons)   # {1, 2, 3, 4}

# Ensemble vide     ATTENTION !
vide_correct = set()       #  Ensemble vide
vide_faux    = {}          #  Dictionnaire vide !
print(type(vide_correct))  # <class 'set'>
print(type(vide_faux))     # <class 'dict'>

# Taille
print(len(fruits))   # 4
```

Erreur fréquente

Pour créer un ensemble vide, on utilise `set()` et **jamais** `{}`. Les accolades vides créent un **dictionnaire** vide, pas un ensemble. C'est l'une des confusions les plus

fréquentes entre ces deux structures.

Propriétés fondamentales des sets

Unicité et non-ordonnancement

```
# 1. UNICITE      pas de doublons possibles
emails = set()
emails.add("alice@example.com")
emails.add("bob@example.com")
emails.add("alice@example.com")    # doublon ignore !
print(emails)    # 2 elements seulement
print(len(emails))    # 2

# 2. NON ORDONNE      pas d'index
notes = {15, 18, 12, 9, 17}
#  notes[0]       TypeError: 'set' object is not subscriptable

# 3. HASHABLE      les elements doivent etre immuables
valide   = {1, "Python", (10, 20), True}    #
# invalide = {[1, 2], {"a": 1}}             #      TypeError

# 4. Verification d'appartenance      tres rapide !
grands_nombres = set(range(1_000_000))
print(999_999 in grands_nombres)    # True      instantane !
```

À retenir

La vérification `in` sur un set est **extrêmement rapide** — temps constant O(1) — quelle que soit la taille de l'ensemble. Sur une liste, cette vérification est linéaire O(n). Pour tester des millions d'appartenances, le set est des centaines de fois plus rapide que la liste.

Ajouter et supprimer des éléments

add(), remove(), discard(), pop(), clear()

```
langages = {"Python", "JavaScript", "Java"}

# add()      ajouter un element
langages.add("Rust")
langages.add("Python")    # deja present      ignore
print(langages)

# remove()      supprimer (leve KeyError si absent)
langages.remove("Java")
```

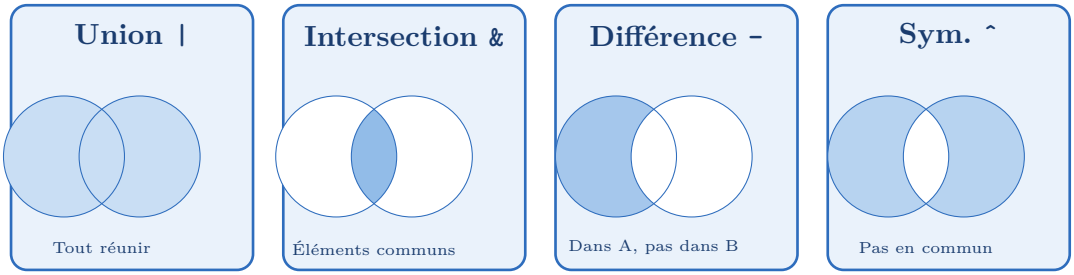
```python
# discard()      supprimer SANS erreur si absent
langages.discard("Java")       # deja supprime      pas d'erreur
langages.discard("Cobol")      # absent      pas d'erreur non
    plus

# pop()      supprimer un element ALEATOIRE
# (non ordonne      on ne sait pas lequel !)
element = langages.pop()
print(f"Supprime : {element}")

# update()      ajouter plusieurs elements
langages.update(["Go", "Kotlin", "Swift"])
print(langages)

# clear()      vider l'ensemble
langages.clear()
print(langages)    # set()
```

Ce qui m'a résisté

La différence entre `remove()` et `discard()` m'a semblé subtile au premier abord — mais elle est importante. `remove()` lève une `KeyError` si l'élément est absent, `discard()` l'ignore silencieusement. Dans la plupart des cas pratiques où on n'est pas certain que l'élément existe, `discard()` est le choix le plus sûr. C'est le même principe que `get()` pour les dictionnaires — toujours préférer la version défensive.

Les opérations ensemblistes

C'est ici que les sets révèlent toute leur puissance — les opérations mathématiques sur les ensembles.



Opérations ensemblistes en action

```python
python_devs = {"Alice", "Bob", "Charlie", "Diana"}
js_devs     = {"Bob", "Diana", "Eve", "Frank"}

# Union      tous les developpeurs
tous = python_devs | js_devs
print(tous)
# {'Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank'}
```

```python
# Aussi avec union()
tous2 = python_devs.union(js_devs)

# Intersection      maitrisent les DEUX langages
full_stack = python_devs & js_devs
print(full_stack)    # {'Bob', 'Diana'}

# Difference      Python seulement (pas JS)
python_only = python_devs - js_devs
print(python_only)    # {'Alice', 'Charlie'}

# Difference symetrique      dans l'un OU l'autre, pas les deux
specialistes = python_devs ^ js_devs
print(specialistes)    # {'Alice', 'Charlie', 'Eve', 'Frank'}

# Operateurs d'assignation
python_devs |= {"Grace"}    # Ajouter a l'ensemble
python_devs -= {"Alice"}    # Retirer de l'ensemble
```

Les relations entre ensembles

Sous-ensemble, sur-ensemble, disjoints

```python
a = {1, 2, 3}
b = {1, 2, 3, 4, 5}
c = {6, 7, 8}

# issubset()      a est-il inclus dans b ?
print(a.issubset(b))      # True  (a      b)
print(a <= b)             # True  (meme chose)
print(a < b)              # True  (sous-ensemble strict)

# issuperset()      b contient-il a ?
print(b.issuperset(a))    # True  (b      a)
print(b >= a)             # True
print(b > a)              # True  (sur-ensemble strict)

# isdisjoint()      aucun element en commun ?
print(a.isdisjoint(c))    # True  (aucun element commun)
print(a.isdisjoint(b))    # False (1, 2, 3 en commun)

# Egalite de deux ensembles
d = {3, 1, 2}
print(a == d)    # True  (meme elements, ordre sans importance)
```

La set comprehension

Comme les listes et les dictionnaires, les sets ont leur syntaxe de compréhension.

Set comprehension

```python
# Syntaxe : {expression for element in iterable if condition}

# Carres uniques
carres = {x**2 for x in range(-5, 6)}
print(carres)
# {0, 1, 4, 9, 16, 25}      pas de doublons !

# Lettres uniques d'une phrase
phrase = "python est un langage puissant"
lettres_uniques = {c for c in phrase if c != " "}
print(len(lettres_uniques))   # nombre de lettres distinctes

# Filtrer les pairs
pairs = {n for n in range(1, 21) if n % 2 == 0}
print(pairs)   # {2, 4, 6, 8, 10, 12, 14, 16, 18, 20}

# Extraire les domaines d'emails
emails = [
    "alice@gmail.com",
    "bob@yahoo.com",
    "charlie@gmail.com",
    "diana@outlook.com"
]
domaines = {email.split("@")[1] for email in emails}
print(domaines)   # {'gmail.com', 'yahoo.com', 'outlook.com'}
```

Le frozenset — Ensemble immuable

`frozenset` est la version **immuable** du set — on ne peut ni ajouter ni supprimer d'éléments après sa création.

frozenset — ensemble immuable

```python
# Creer un frozenset
jours_feries = frozenset({"Noel", "Nouvel An", "Paques"})
print(jours_feries)

#     Impossible de modifier
# jours_feries.add("Ascension")       AttributeError

#     Utilisable comme cle de dictionnaire
planning = {
    frozenset({"lundi", "mercredi"}): "Cours Python",
    frozenset({"mardi", "jeudi"}):    "Cours Maths"
}

#     Utilisable comme element d'un autre set
```

```python
ensemble_de_sets = {
    frozenset({1, 2}),
    frozenset({3, 4}),
    frozenset({1, 2})   # doublon ignore
}
print(len(ensemble_de_sets))   # 2

# Toutes les operations de lecture fonctionnent
print("Noel" in jours_feries)          # True
print(len(jours_feries))               # 3
```

Cas pratiques — Dédoublonnage et comparaisons

Supprimer les doublons d'une liste

```python
# M thode classique et efficace
notes = [15, 18, 12, 18, 9, 15, 17, 12, 18]
notes_uniques = list(set(notes))
print(notes_uniques)   # [9, 12, 15, 17, 18] (ordre non garanti
    )

# Preserver l'ordre (Python 3.7+ avec dict)
notes_uniques_ordonn = list(dict.fromkeys(notes))
print(notes_uniques_ordonn)   # [15, 18, 12, 9, 17]

# Trouver les elements communs entre deux listes
liste_a = [1, 2, 3, 4, 5, 6]
liste_b = [4, 5, 6, 7, 8, 9]
communs = list(set(liste_a) & set(liste_b))
print(communs)   # [4, 5, 6]

# Verifier si deux listes ont des elements en commun
def ont_elements_communs(l1, l2):
    return not set(l1).isdisjoint(set(l2))

print(ont_elements_communs([1, 2, 3], [3, 4, 5]))   # True
print(ont_elements_communs([1, 2, 3], [7, 8, 9]))   # False
```

Analyse de données avec les sets

```python
# Quels utilisateurs ont visite les deux pages ?
visiteurs_accueil = {"Alice", "Bob", "Charlie", "Diana", "Eve"}
visiteurs_contact  = {"Bob", "Diana", "Frank", "Grace"}

# Ont visite les deux
les_deux = visiteurs_accueil & visiteurs_contact
print(f"Les deux pages : {les_deux}")
```

```python

# N'ont visité QUE l'accueil
accueil_seul = visiteurs_accueil - visiteurs_contact
print(f"Accueil seulement : {accueil_seul}")

# Total de visiteurs uniques
total = visiteurs_accueil | visiteurs_contact
print(f"Total visiteurs uniques : {len(total)}")
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `{}` ne crée pas un set vide — mais un dict vide. Cette confusion entre accolades de set et accolades de dict m'a piégé dès le début. Python utilise `{}` pour les deux structures, mais sans contenu, il choisit le dictionnaire par défaut. La seule façon de créer un set vide est `set()`. Un détail minuscule qui peut créer des bugs silencieux et difficiles à tracer.

2. La vérification `in` est instantanée sur un set. Sur une liste de un million d'éléments, `x in liste` peut prendre du temps. Sur un set de un million d'éléments, `x in ensemble` est **instantané** — temps constant, quelle que soit la taille. Cette différence de performance m'a ouvert les yeux sur l'importance de choisir la bonne structure de données selon le cas d'usage.

3. Les opérateurs `|`, `&`, `-`, `^` sur les sets. Voir des opérateurs mathématiques directement applicables à des collections Python m'a rappelé la théorie des ensembles de mes cours de maths. Python traduit directement des concepts mathématiques abstraits en code concret et lisible. C'est l'une des preuves que Python a été conçu par des gens qui pensent autant aux mathématiques qu'à l'informatique.

Résumé du Chapitre 11

Ce que j'ai appris dans ce chapitre :

- Un set est une collection **non ordonnée** d'éléments **uniques**.
- Créer avec `{1,2,3}` ou `set()` — jamais `{}` pour le vide.
- `add()`, `discard()` — ajouter et supprimer sans erreur.
- **Union `|`, Intersection `&`, Différence `-`, Sym. `^`.**
- `issubset()`, `issuperset()`, `isdisjoint()` — relations entre sets.
- **Set comprehension** : `{expr for x in iter if cond}`.
- `frozenset` — version immuable, utilisable comme clé de dictionnaire.

Vérification `in` en temps constant O(1) — bien plus rapide que les listes.

"Un ensemble ne retient que l'essentiel. Pas de répétition, pas de bruit — juste l'unicité."